

Song Genre Prediction

Team 6

Avani Gandhi^[1937923], Chintan Vekariya^[2034517], Laura Schulz^[1682482], Manasi Patil^[2034414], Priyanka Gangavkar^[1981729], and Seowon Jeong^[2034736]

University of Mannheim, Mannheim, Germany

1 Introduction

Music holds a significant place in the lives and preferences of individuals. People are particularly good at listening to short parts of a song to identify different aspects of a song, including the artist, the title, and even the genre.

In today's digital era, music services have collected an enormous amount of audio data spanning a wide range of genres and artists [7]. To organize this data and generate personalized recommendations, it is crucial to develop a classification system identifying the genre of a song.

Previous research has explored various approaches to address this. Spectrograms derived from audio signals have shown promise in effectively classifying genres [2]. As well as genre classification frameworks for music files, based on a language modeling approach [5].

In contrast to these approaches, our Data Mining Project aims to train a model that can predict the genre of a song based on the given dataset. Therefore we want to identify the most important features and best-fit classification method.

Before training the models, several tasks, such as data preprocessing, will be performed. In the first trial of model training we will use various classification algorithms. Based on that, we will perform hyperparameter tuning and derive the final ensemble model.

2 Data Understanding

The dataset we will use to train our model is the "Audio features and lyrics of Spotify songs" [4]. The dataset originally comes from the TidyTuesday Project [1], and contained songs and their metadata stored in Spotify [9], a music streaming service. A Kaggle user added lyrics to a subset of the songs using the Genius Library in R (discontinued in 2022). We picked this dataset as it provided us not only song lyrics but also some other technical variables about the songs.

The Kaggle dataset consists of 25 features and 18454 songs, taking up 43 MB of storage. The features are described in detail in Table 1. Our target variable is **playlist_genre**, which has 6 classes: *pop*, *rock*, *rap*, *r&b*, *latin* and *edm*. The class distribution of the variable is shown in Figure 1. As shown in the Bar

Table 1. Features in the dataset and their descriptions

Feature Name	Description
track_id	Unique Song ID
track_name	Song Name
track_artist	Song Artist
lyrics	Full Lyric text
track_popularity	A measure of the song's popularity (0-100)
track_album_id	Unique Album ID
track_album_name	Album Name
track_album_release_date	Album Release Date
playlist_name	Name of Playlist the song was placed on
playlist_id	Unique Playlist ID
playlist_genre	Genre of Playlist
playlist_subgenre	Subgenre of Playlist
danceability	A measure of how suitable a song is for dancing (0.0 - 1.0)
energy	A measure of intensity and activity on the song (0.0 - 1.0)
key	The estimated overall key of the track. Integers map to pitches using standard Pitch Class notation. If no key was detected, the value is -1.
loudness	The overall loudness of a track in dB
mode	Modality (major or minor) of a track (Major - 1, Minor - 0)
speechiness	Measure of relative amount of spoken words on a song (0.0 - 1.0)
acousticness	Confidence measure of whether the song is acoustic (0.0 - 1.0)
instrumentalness	Confidence measure of whether a song contains no vocals (0.0 - 1.0)
liveness	Confidence measure of whether the song was recorded live (0.0 - 1.0)
valence	A measure describing the musical positiveness conveyed by a track (0.0 - 1.0)
tempo	The overall estimated tempo of a track in beats per minute (BPM)
duration_ms	Duration of song in ms
language	Language of the lyrics

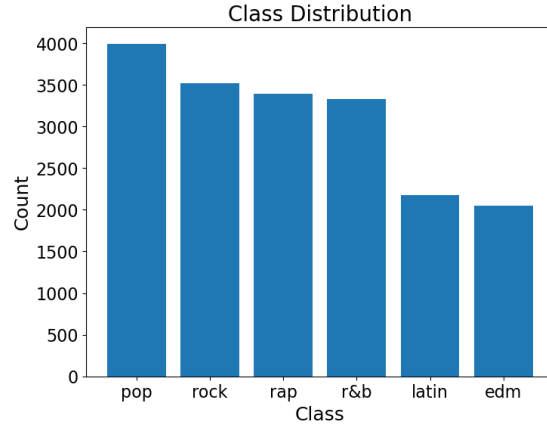


Fig. 1. Initial Class Distribution of the `playlist_genre` variable

plot, the classes are unbalanced, with the genres *latin* and *edm* being the least represented.

Of all the features in the dataset, 11 are continuous and numerical. We performed a statistical analysis of these features to gain insights on their value ranges and distribution. The results of this initial analysis is in Table 2, with key details summarized below.

- **track_popularity:** The measure of popularity of a song presents as integer values between 1 and 100. The average of the values sits at roughly 42, while the median at 48 and the more frequent half of the values are within the range of 24 to 62. This feature also contains 1647 songs for which the popularity is 0. The remaining values follow a normal distribution.
- **danceability, energy, valence:** These variables are Spotify-assigned values giving the danceability, energy and positivity of a track a value between 0.0 and 1.0. They follow a normal distribution, averaging 0.64, 0.69 and 0.52 respectively. The features danceability and energy are not very widely distributed, with most of the values between 0.55 and 0.8, indicating the songs in the dataset being mostly high-energy and easy to dance to. The valence variable is more widely distributed, with a median of 0.52 and the 25th and 75th percentile intervals being 0.34 and 0.7. The songs in the dataset thus vary greatly in terms of positivity.
- **acousticness, instrumentalness, liveness:** These variables are confidence measures of whether a song is acoustic, has no vocals and was recorded live respectively. All three variables are extremely positively skewed or right skewed distributions, with the majority of the values being quite small. This can be seen in the 75th percentile values of the variables, with acousticness and liveness having a value of roughly 0.25, while instrumentalness is even smaller at 0.0017.

- **speechiness**: The speechiness feature is a relative measure of how many spoken words are in a song. The more purely a file is simply spoken words, the closer the value is to 1.0. As this dataset comprises songs, the values are expected to be lower. This is reflected in the mean and median being approx. 0.10 and 0.06. However, there are still songs with a higher speechiness value present, like raps, and thus the max value in the dataset is a speechiness of approximately 0.92.
- **loudness, tempo, duration_ms**: These continuous variables describe objective technical measures of the song. The loudness of a song is averaged across the track, and is useful for comparing relative loudness, as it has negative values ranging between 0 and -60 dB. From the percentile values it can be deduced most of the songs are louder compared to the least loud song in the dataset. The tempo is measured in beats-per-minute (BPM) and is normally distributed with an average of approx. 121 BPM and a standard deviation of approx. 26 BPM. The duration_ms values range between 31893 and 517810 which corresponds to roughly 0.5 to 8.5 minutes. The mean and the median values correspond to 3.8 and 3.7 minutes respectively.

An initial correlation analysis performed using multinomial logistic regression (after normalizing the numerical features) produced some insights into the correlation of the individual numerical variables on the different genres. The resulting regression coefficients are visualized as a heatmap in Figure 2. As seen in the figure, **danceability** seems to have a strong negative correlation for rock songs, while **speechiness** has a strong positive correlation with rap songs. However, most of the numerical features appear not to have a strong correlation with the genre, as most of the coefficient values are close to 0.

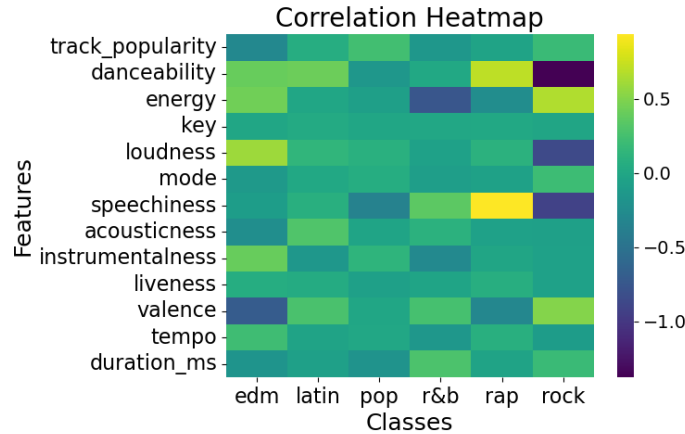


Fig. 2. Regression Coefficient per genre Heatmap

As the dataset was initially created for a data mining competition, it underwent some minimal data quality checking. Upon inspection of the data, there exist some purely instrumental tracks for which the lyric field is null.

Table 2. Statistics of the numerical features

	count	mean	std	min	25%	50%	75%	max
track_popularity	18454	42.4388	24.6167	0.0000	24.0000	48.0000	62.0000	100.000
danceability	18454	0.6441	0.1501	0.0985	0.5460	0.6600	0.7550	0.979
energy	18454	0.6923	0.1806	0.0167	0.5730	0.7150	0.8350	1.000
loudness	18454	-6.7692	2.9208	-34.2830	-8.2490	-6.2270	-4.7190	1.275
speechiness	18454	0.1062	0.1023	0.0224	0.0397	0.0603	0.1300	0.918
acousticness	18454	0.1753	0.2178	0.0000	0.0161	0.0819	0.2540	0.992
instrumentalness	18454	0.0512	0.1683	0.0000	0.0000	0.0000	0.0017	0.987
liveness	18454	0.1896	0.1538	0.0094	0.0927	0.1280	0.2460	0.996
valence	18454	0.5206	0.2287	0.0000	0.3450	0.5220	0.7000	0.991
tempo	18454	120.8122	27.5864	37.1140	98.8560	120.0450	135.9840	214.047
duration_ms	18454	230319	57255	31893	193230	221340	258078	517810

3 Preprocessing and Text Mining

Before training various classifiers, the data needed to be preprocessed. In this section we detail the various actions we performed on the data.

3.1 Preprocessing Steps

Our first point of focus was selecting relevant data from the original dataset.

1. **Data Selection:** The dataset contains songs in many different languages. For the purposes of our task, we decided to restrict the songs we wanted our classifiers to be trained by to just be English language songs. Thus we removed the rows in the dataset that were labeled as Non-English songs, using the **language** feature.
2. **Handling Missing Values:** To focus on only songs with lyrics, we removed the 154 songs that were purely instrumental and had an empty cell in the **lyrics** column.
3. **Initial Feature Selection:** Relevant columns necessary for the analysis were selected while excluding irrelevant or redundant features like IDs and names of the songs, albums and playlists. This streamlined the dataset and focused the analysis on the most pertinent information. The features selected were: **lyrics**, **track_popularity**, **playlist_genre**, **danceability**, **energy**, **key**, **loudness**, **mode**, **speechiness**, **acousticness**, **instrumentalness**, **liveness**, **valence**, **tempo** and **duration_ms**.

4. **Prevent Feature Duplication:** We added `_F` to the end of the names of the numerical features to prevent overlap with words appearing in lyrics.
5. **Encoding target variable:** The target variables which were categorical values, were encoded to integer values.
6. **Normalization:** To improve the quality of the classifiers and prevent the variables with larger ranges from dominating the classification decisions, normalisation was performed on the unnormalized features: `track_popularity`, `loudness`, `tempo`, and `duration_ms`.

3.2 Text Mining

In addition to preprocessing the numerical variables, we used several text mining techniques to extract features from the `lyrics` feature.

1. **Tokenization:** The lyrics were tokenized into individual words or tokens. To prevent capitalization from creating near-identical tokens, the text was all changed to lowercase beforehand. The tokenization and the other following operations were performed using the NLTK package [6].
2. **Stopword Removal:** Common stopwords, such as ‘the’, ‘is’, and ‘and’ were identified and removed from the tokenized lyrics.
3. **Stemming:** To make sure words like ‘dance’ and ‘dancer’ were not treated as separate words, stemming, which reduces words to their root forms, was performed on the tokens.
4. **Pruning:** Unnecessary words like numbers and months were pruned from the lyrics to improve the quality of the text data.
5. **Vectorization:** Song lyrics often repeat words that are heavily correlated with the subject matter of the song, as well as words that occur in the chorus. To make sure this aspect is reflected in our model, we used the TF-IDF vectorizing technique. TF-IDF assigns weights to words based on their frequency in the document and inverse frequency across all documents, highlighting terms that are more indicative of the document’s content. We limited the number of generated features to manage dimensionality and computational complexity, as the initial number of generated features was over 25,000.

In order to not only train classifiers based on the frequencies of single words, but also compare their performance to classifiers trained on single words and word pairs, two versions of preprocessed data were created. The dataset with only single extracted words was limited to 4000 most relevant features, while the additional dataset was created with a limit of 10000 most relevant features.

4 Classification Algorithms and Evaluation

4.1 Classification Algorithms

Based on the exploration and preprocessing of the dataset we decided to train different models with the following algorithms. Their results are visualised in Figure 3

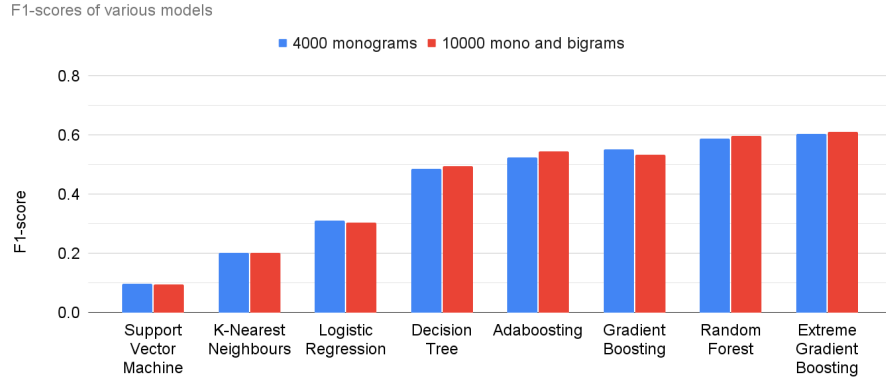


Fig. 3. F1 Scores of models (pre-tuning)

Logistic Regression We employ logistic regression assuming that the probability of a song belonging to a certain genre can be modeled as a linear combination of features extracted from the lyrics data.

Approach Utilizing scikit-learn [8] in Python, we conduct logistic regression with multinomial logistic regression strategies to classify songs into multiple genres. We then analyze the confusion matrix to assess classification performance.

Evaluation As seen in the figure, the F1 score achieved for 4000 monograms was 0.31, indicating a moderate balance between precision and recall. Interestingly, the F1 score for 10000 features including bigrams was slightly lower at 0.30, suggesting that the model's performance decreased when considering pairs of consecutive words.

Decision Trees The intention is to utilize decision trees to partition the feature space based on lyrics features, creating a hierarchical structure for genre classification.

Approach Implement decision trees using scikit-learn [8], allowing for interpretable models and insights into feature importance.

Evaluation When employing the Decision Tree algorithm for song genre prediction, we achieved an F1 score of 0.48 when using monograms as features and a slightly improved F1 score of 0.49 when considering bigrams as well. This marginal increase in F1 score for bigrams suggests that incorporating sequences of two words at a time enhanced the algorithm's ability to capture nuanced patterns and improve classification accuracy in the context of genre prediction based on song lyrics.

K-Nearest Neighbours Use KNN to classify songs based on the similarity of their lyrics features to those of neighboring songs in the feature space.

Approach Implement KNN using scikit-learn [8], selecting the optimal number of neighbors k .

Evaluation As seen in the figure, the F1 score achieved for 4000 monograms was 0.31, indicating a moderate balance between precision and recall. Interestingly, the F1 score for 10000 monograms and bigrams was slightly lower at 0.30, suggesting that the model's performance decreased when considering pairs of consecutive words.

Support Vector Machines The intention is to utilize SVM to classify songs into different genres by finding the optimal hyperplane that separates them in the feature space.

Approach Implement SVM using scikit-learn [8], selecting appropriate kernels e.g., linear based on the data distribution.

Evaluation An F1 score of 0.097 for monograms and 0.096 for bigrams indicates poor classification performance for both feature types when using the SVM algorithm. These scores suggest that the model struggles to accurately classify songs into their respective genres based on the provided features.

Random Forest We employ random forests to improve upon decision tree performance by building an ensemble of trees and reducing overfitting.

Approach Use scikit-learn [8] to build a random forest classifier, leveraging the power of ensemble learning for genre prediction.

Evaluation When employing the decision tree algorithm for song genre prediction, we observed an F1 score of 0.58 when using monograms as features, and a slightly improved F1 score of 0.59 when utilizing bigrams. This marginal increase suggests that considering sequences of two words at a time, rather than single words alone, yielded a slight enhancement in the classification accuracy of the model.

Gradient Boosting We apply gradient boosting to iteratively improve classification accuracy by combining multiple weak learners.

Approach Utilize gradient boosting algorithms using scikit-learn [8], which offer efficient implementations and potential performance gains.

Evaluation When employing the gradient boosting algorithm, we obtained an F1 score of 0.55 for monograms and 0.53 for bigrams. This indicates that the model achieved slightly higher classification accuracy when considering only individual words compared to pairs of consecutive words as well.

Extreme Gradient Boosting Employ XGBoost to capture non-linear relationships between lyrics features and genre labels, potentially improving classification accuracy. XGBoost is better at generalisation than gradient boosting, which is due to its more advanced regularization.

Approach Utilize the XGBoost’s python library [3] to build an ensemble of decision trees, sequentially optimizing for errors in classification.

Evaluation The F1 score for monograms was 0.604, while for bigrams, it slightly increased to 0.609. This marginal improvement suggests that incorporating sequential pairs of words enhances the algorithm’s ability to capture nuanced patterns in the text data, leading to a modest enhancement in classification accuracy.

Adaptive Boosting We apply AdaBoost to iteratively improve classification accuracy by combining multiple weak learners, potentially enhancing model performance. Its ensemble building approach differs slightly from Gradient Boosting, as it adjusts the weights of the samples and forms the model also by taking the individual weights into account.

Approach Utilize AdaBoostClassifier from scikit-learn [8], integrating weak learners such as decision trees for genre prediction.

Evaluation We observed an F1 score of 0.52 when utilizing monograms and a slightly improved F1 score of 0.54 when incorporating dual grams. This suggests that the inclusion of sequential word pairs provided a modest enhancement in classification accuracy compared to the singular word features.

4.2 Optimisation

We tried various approaches to further improve our results.

Stacking Classifier After evaluating various classifiers for genre prediction, we combined the top 5 performing models using stacking techniques. This approach allowed us to leverage the strengths of individual classifiers while optimizing overall performance, resulting in a robust genre prediction system tailored to song data. We used the scikit-learn library to implement the Stacking Classifier, which combines the ensemble using Logistic Regression. As an example of the improvement due to this technique, the final f1-score of the ensemble model made up of the five best models shown in Figure 3 was 0.62, which is an increase of roughly 0.011 compared to the best performing XGBoost model.

Hyperparameter Tuning To further improve model performance, we used some manual hyperparameter turning, Randomised Search and Grid Search (the latter two using a 5 split cross evaluation) to find the best hyperparameter combinations for our five best performing models. By improving the performance of the individual classifiers (e.g. the Gradient Boosted Model’s F1 score was increased from around 0.53 to nearly 0.62), we attempted to improve the performance of the final stacking classifier as well.

4.3 Critical Evaluation

Our model has the following shortcomings, which can impact its performance.

Fundamental Issues with the Dataset There were several fundamental issues with the dataset we used. Since we only considered songs labeled in English, the number of songs in Latin genres drastically decreased. This led to a decrease in predictive performance for Latin genres, dragging down the overall average F1 score of the model. Moreover, we did not perform Data Balancing, because the model’s performance declined when using balanced data and we could not determine the cause.

Resource limitations Due to constraints related to computational performance, we were unable to implement the approaches we had planned. Specifically, the initial number of features was excessively large, requiring us to limit it to 10,000 based on their frequency. However, since the preprocessed, simplified dataset was also substantial, we could not attempt to adjust the split ratio between the training set and the test set, as well as the split ratio in cross evaluation. In the same context, correlation based feature selection could not be performed. Furthermore, when attempting to apply Grid Search to several models (e. g. the Gradient Boosted Model) the execution time exceeded one day. In such cases we had to conduct manual tuning.

5 Conclusion

To conclude, we trained eight models and the stacking classifier to get an understanding of which model the best performing genre prediction would be. The preprocessing of the data was the most important part, as we needed to get an understanding of dataset and an first idea of which features might be the most relevant to train the models. Then we iteratively tried out various algorithms with different hyperparameter combinations.

In the end the best performing model was the Gradient Boosted Classifier with an F1 score of roughly 0.62. Due to resource limitations, we attempted to prefit the models instead of using the classifier’s inbuilt training with cross-evaluation, however this stacked classifier ended up performing worse than the individually trained models. As the stacked classifier was being trained on the same data as the individual models, it is likely that the trained model was overfitted and thus performed worse.

Upon comparing the feature importances of the resulting best models with the initial feature correlation, we found a lot of similarities. In particular the numerical features like **danceability** and **tempo** are among the most relevant features to determine the genre of a song. Additionally, some of the most relevant lyric words were several slurs, curses and words related to love. As the first two appear frequently and distinctly in rap song lyrics, they ended up being the most relevant for models to classify the song. While our analysis focused only on overall most relevant features, further research can be undertaken to determine the most relevant features per genre, perhaps including a bigger and more balanced dataset. Additionally, more advanced model algorithms like Neural Networks can also be applied to deal the large number of features and rows.

References

1. Community, D.S.L.: Spotify songs (2020), <https://github.com/rfordatascience/tidytuesday/tree/master/data/2020/2020-01-21>
2. Costa, Y.M., Oliveira, L.S., Silla, C.N.: An evaluation of convolutional neural networks for music classification using spectrograms. *Applied Soft Computing* **52**, 28–38 (2017). <https://doi.org/https://doi.org/10.1016/j.asoc.2016.12.024>, <https://www.sciencedirect.com/science/article/pii/S1568494616306421>
3. xgboost developers: Xgboost python package (2022), <https://xgboost.readthedocs.io/en/stable/python/index.html>
4. Kaggle: Audio features and lyrics of spotify songs (2020), <https://www.kaggle.com/datasets/imuhammad/audio-features-and-lyrics-of-spotify-songs>
5. Langlois, T., Marques, G.: Automatic music genre classification using a hierarchical clustering and a language model approach. In: 2009 First International Conference on Advances in Multimedia. pp. 188–193 (2009). <https://doi.org/10.1109/MMEDIA.2009.42>, <https://ieeexplore.ieee.org/document/5206888>
6. NLTK: nltk.tokenize package, <https://www.nltk.org/api/nltk.tokenize.html>
7. O’Brien, T.: Learning to understand music from shazam (2017), <https://medium.com/@tsob/learning-to-understand-music-from-shazam-56a60788b62f>
8. Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., Duchesnay, E.: Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research* **12**, 2825–2830 (2011), <https://scikit-learn.org/stable/index.html>
9. Spotify: Spotify web player, <https://open.spotify.com/>